

Portfolio Data Labeling

About This Portfolio

This portfolio showcases my expertise in **Data Labeling** across both **Text** and **Image** domains. It demonstrates not only annotation skills but also advanced **Quality Assurance (QA)** processes, dataset structuring, and export validation.

Key Highlights

- **Text Labeling:**
 - Text Classification, Named Entity Recognition (NER), Intent Classification, Topic Labeling.
 - QA processes ensuring accuracy, consistency, and completeness.
 - Export and validation in JSON/CSV formats.
- **Image Labeling:**
 - Image Classification, Object Detection (Bounding Boxes), Image Segmentation (Semantic, Instance, Panoptic), Keypoint Annotation (Pose Estimation).
 - QA processes across all annotation types.
 - Dataset export and structuring in COCO, Pascal VOC, and YOLO formats.

Competencies Demonstrated

- Strong attention to detail and guideline compliance.
- Ability to handle edge cases and ambiguous scenarios.
- Versatility across NLP and Computer Vision projects.
- Professional dataset preparation for machine learning pipelines.

Part I – Text Labeling

Chapter: Text Classification (Detailed)

1. Definition and Purpose

- **Text Classification** is the process of assigning predefined categories to text samples.
- It is used in tasks such as:
 - **Sentiment Analysis** (Positive, Negative, Neutral).
 - **Spam Detection** (Spam, Not Spam).
 - **Product Review Categorization** (Electronics, Clothing, Food).
 - **Customer Support Routing** (Billing, Technical Issue, General Inquiry).
- The goal is to ensure that each text is consistently mapped to the correct category.

2. Task Setup

- **Label Schema Creation:**
 - Define categories clearly and make them mutually exclusive.
 - Example: Sentiment → Positive, Negative, Neutral.
- **Annotation Tool Configuration:**
 - Import text samples into the platform (Labelbox, CVAT for text, or internal tool).
 - Choose between **single-label** (one category per text) or **multi-label** (multiple categories per text).
- **Guidelines Document:**
 - Provide definitions for each category.
 - Include examples of correct vs. incorrect classification.
 - Clarify how to handle ambiguous cases (e.g., sarcasm, mixed sentiment).

3. Annotation Process

- **Step 1: Read the text carefully** → understand context and meaning.
- **Step 2: Identify the main signal** → sentiment, spam indicators, or topic keywords.
- **Step 3: Assign the correct label** → based on schema and guidelines.
- **Step 4: Double-check consistency** → similar texts should receive the same label.

Examples:

- “I love this phone, it works perfectly.” → **Positive**
- “Worst service ever, I will never come back.” → **Negative**
- “Click here to claim your free prize.” → **Spam**
- “Your invoice is ready for download.” → **Billing Inquiry**

4. Quality Assurance (QA) in Classification

- **Error Types:**
 - **Misclassification:** assigning the wrong label (e.g., labeling “Worst service ever” as Positive).
 - **Inconsistency:** similar texts labeled differently.
 - **Over-labeling:** assigning multiple categories when only one is required.
 - **Under-labeling:** leaving text unlabeled.
- **Correction Steps:**
 - Review dataset samples systematically.
 - Compare labels across similar texts.
 - Correct misclassified or inconsistent labels.
 - Document changes for transparency.

5. Export and Validation

- **Export Format:** JSON or CSV.
- **Validation Checks:**
 - Each text sample must have at least one label.
 - Labels must match the predefined schema.
 - No empty or invalid categories.

Example JSON Export:

```
{  
  "text": "Worst service ever, I will never come back.",  
  "label": "Negative"  
}
```


Chapter: Named Entity Recognition (NER) (Detailed)

1. Definition and Purpose

- **Named Entity Recognition (NER)** is the process of identifying and labeling specific entities in text.
- Common entity categories include:
 - **PERSON** → names of people.
 - **LOCATION** → cities, countries, landmarks.
 - **ORGANIZATION** → companies, institutions, agencies.
 - **DATE** → explicit dates or time expressions.
 - **PRODUCT** → items, brands, or services.
- Purpose: to structure unorganized text into machine-readable data for tasks like search, QA systems, and knowledge graphs.

2. Task Setup

- **Define Entity Schema:**
 - Decide which entity types are required (e.g., PERSON, LOCATION, ORGANIZATION).
 - Ensure categories are mutually exclusive (e.g., “Microsoft” must always be ORGANIZATION, never PERSON).
- **Annotation Tool Configuration:**
 - Import text samples into the annotation platform.
 - Enable sequence labeling mode (highlighting words/phrases).
- **Guidelines Document:**
 - Provide definitions and examples for each entity type.
 - Clarify edge cases (e.g., “Apple” → PRODUCT vs. ORGANIZATION depending on context).

3. Annotation Process

- **Step 1: Read the text carefully** → understand context.
- **Step 2: Identify entities** → highlight words/phrases that match schema.
- **Step 3: Assign correct label** → PERSON, LOCATION, ORGANIZATION, etc.
- **Step 4: Double-check consistency** → same entity must always have the same label across dataset.

Examples:

- “John works at Microsoft in Paris.”
 - John → PERSON
 - Microsoft → ORGANIZATION
 - Paris → LOCATION
- “The iPhone 15 was released in 2023.”
 - iPhone 15 → PRODUCT
 - 2023 → DATE

4. Quality Assurance (QA) in NER

- **Error Types:**
 - **Missing entities** → forgetting to label “Paris” as LOCATION.
 - **Incorrect labels** → labeling “Microsoft” as PERSON instead of ORGANIZATION.
 - **Inconsistent labeling** → “NYC” labeled as LOCATION in one sample, ORGANIZATION in another.
- **Correction Steps:**
 - Review text samples systematically.
 - Compare labels across similar contexts.
 - Correct mislabeling and ensure consistency.
 - Document changes for transparency.

5. Export and Validation

- **Export Format:** JSON or CSV.
- **Validation Checks:**
 - Each entity must be linked to its category.
 - No empty or invalid labels.
 - Consistency across dataset.

Example JSON Export:

```
{  
  "text": "John works at Microsoft in Paris.",  
  "annotations": [  

```

```
{"entity": "John", "label": "PERSON"},  
{"entity": "Microsoft", "label": "ORGANIZATION"},  
{"entity": "Paris", "label": "LOCATION"}  
]  
}
```

Chapter: Intent Classification (Detailed)

1. Definition and Purpose

- **Intent Classification** is the process of identifying the underlying purpose or goal behind a user's text or query.
- Common use cases:
 - **Customer Support** → "I need help resetting my password." → Support Intent.
 - **Travel Booking** → "Book me a flight to London." → Travel Intent.
 - **Information Retrieval** → "What's the weather tomorrow?" → Weather Query Intent.
 - **E-commerce** → "Add iPhone 15 to my cart." → Purchase Intent.
- Purpose: to route queries to the correct system or department and enable automation (chatbots, voice assistants, etc.).

2. Task Setup

- **Define Intent Categories:**
 - Travel Booking, Weather Query, Customer Support, Purchase, General Inquiry.
- **Annotation Tool Configuration:**
 - Import text samples into the annotation platform.
 - Choose **single-label classification** (one intent per query).
- **Guidelines Document:**
 - Provide definitions for each intent.
 - Include examples of correct vs. incorrect intent assignment.
 - Clarify edge cases (e.g., "Can you book me a flight and tell me the weather?" → multi-intent scenario).

3. Annotation Process

- **Step 1: Read the text carefully** → understand the user's request.
- **Step 2: Identify the main action or goal** → booking, asking, buying, requesting help.
- **Step 3: Assign the correct intent label** → based on schema.
- **Step 4: Double-check consistency** → similar queries must receive the same intent.

Examples:

- “Book me a flight to London.” → **Travel Booking Intent**
- “What’s the weather tomorrow?” → **Weather Query Intent**
- “I need help with my invoice.” → **Customer Support Intent**
- “Add iPhone 15 to my cart.” → **Purchase Intent**

4. Quality Assurance (QA) in Intent Classification

- **Error Types:**
 - **Misclassification** → labeling “Book me a flight” as Weather Query instead of Travel Booking.
 - **Inconsistency** → similar queries labeled differently.
 - **Multi-intent confusion** → queries with multiple goals incorrectly labeled as single intent.
- **Correction Steps:**
 - Review dataset samples systematically.
 - Compare labels across similar queries.
 - Correct misclassified or inconsistent labels.
 - Document edge cases and clarify guidelines.

5. Export and Validation

- **Export Format:** JSON or CSV.
- **Validation Checks:**
 - Each text sample must have exactly one intent label (unless multi-intent is allowed).
 - Labels must match the predefined schema.
 - No empty or invalid categories.

Example JSON Export:

```
{  
  "text": "Book me a flight to London.",  
  "label": "Travel Booking Intent"  
}
```

Chapter: Topic Labeling (Detailed)

1. Definition and Purpose

- **Topic Labeling** is the process of assigning a general subject area or domain to a text sample.
- Common use cases:
 - News categorization (Politics, Sports, Finance, Technology).
 - Content recommendation systems (Health, Entertainment, Education).
 - Document organization in large datasets.
- Purpose: to group texts by subject matter, making them easier to analyze, search, and recommend.

2. Task Setup

- **Define Topic Categories:**
 - Example schema: Politics, Sports, Finance, Technology, Health, Entertainment.
 - Ensure categories are broad enough to cover all samples but specific enough to avoid overlap.
- **Annotation Tool Configuration:**
 - Import text samples into the annotation platform.
 - Choose **single-label classification** (one topic per text) or **multi-label classification** (multiple topics per text if allowed).
- **Guidelines Document:**
 - Provide definitions for each topic.
 - Include examples of correct vs. incorrect topic assignment.
 - Clarify edge cases (e.g., “The government invested in AI research” → Politics + Technology).

3. Annotation Process

- **Step 1: Read the text carefully** → identify the main subject.
- **Step 2: Match keywords and context** → finance terms, sports events, political figures.
- **Step 3: Assign the correct topic label(s)** → based on schema.
- **Step 4: Double-check consistency** → similar texts must receive the same topic.

Examples:

- “The stock market dropped 5% today.” → **Finance**
- “The team won the championship after a tough match.” → **Sports**
- “New AI technology is transforming healthcare.” → **Technology + Health**
- “The president announced new economic reforms.” → **Politics + Finance**

4. Quality Assurance (QA) in Topic Labeling

- **Error Types:**
 - **Misclassification** → labeling “The team won the championship” as Finance instead of Sports.
 - **Inconsistency** → similar texts labeled differently.
 - **Over-labeling** → assigning too many topics when only one is relevant.
 - **Under-labeling** → missing a secondary topic when multi-labeling is required.
- **Correction Steps:**
 - Review dataset samples systematically.
 - Compare topic assignments across similar texts.
 - Correct misclassified or inconsistent labels.
 - Document edge cases and update guidelines.

5. Export and Validation

- **Export Format:** JSON or CSV.
- **Validation Checks:**
 - Each text sample must have at least one topic label.
 - Labels must match the predefined schema.
 - No empty or invalid categories.

Example JSON Export:

```
{  
  "text": "New AI technology is transforming healthcare.",  
  "labels": ["Technology", "Health"]  
}
```


Part II – Image Labeling

Chapter: Image Classification (Detailed)

1. Definition and Purpose

- **Image Classification** is the process of assigning a predefined category to an entire image.
- Unlike object detection or segmentation, classification does not localize objects but determines the **overall class** of the image.
- Common use cases:
 - Medical imaging (X-ray → Healthy vs. Pneumonia).
 - Product categorization (Image → Electronics, Clothing, Furniture).
 - Wildlife monitoring (Photo → Cat, Dog, Horse).
- Purpose: to group images into categories for training machine learning models or organizing datasets.

2. Task Setup

- **Define Classification Schema:**
 - Categories must be clear, mutually exclusive, and cover all possible images.
 - Example schema: Cat, Dog, Horse, Other.
- **Annotation Tool Configuration:**
 - Import images into the annotation platform (CVAT, Labelbox, Roboflow).
 - Select **classification mode** (assigning one label per image).
- **Guidelines Document:**
 - Provide definitions for each category.
 - Include examples of correct vs. incorrect classification.
 - Clarify edge cases (e.g., “Is a cartoon dog labeled as Dog or Other?”).

3. Annotation Process

- **Step 1: Open the image** → observe the main subject.
- **Step 2: Identify the dominant object or theme** → animal, product, medical condition.
- **Step 3: Assign the correct category label** → based on schema.

- **Step 4: Double-check consistency** → similar images must receive the same label.

Examples:

- Image of a cat sitting on a sofa → **Cat**
- Image of a dog running in a park → **Dog**
- Image of a horse in a stable → **Horse**
- Image of a car → **Other**

4. Quality Assurance (QA) in Image Classification

- **Error Types:**
 - **Misclassification** → labeling a dog image as Cat.
 - **Inconsistency** → similar images labeled differently.
 - **Over-labeling** → assigning multiple categories when only one is required.
 - **Under-labeling** → leaving an image unlabeled.
- **Correction Steps:**
 - Review dataset samples systematically.
 - Compare labels across similar images.
 - Correct misclassified or inconsistent labels.
 - Document changes for transparency.

5. Export and Validation

- **Export Format:** JSON or CSV.
- **Validation Checks:**
 - Each image must have exactly one classification label.
 - Labels must match the predefined schema.
 - No empty or invalid categories.

Example JSON Export:

```
{  
  "image": "dog_001.jpg",  
  "label": "Dog"  
}
```


Chapter: Object Detection (Bounding Boxes) (Detailed)

1. Definition and Purpose

- **Object Detection** is the process of identifying and localizing specific objects within an image using bounding boxes.
- Unlike classification (which labels the entire image), detection provides both:
 - **Category** → what the object is (e.g., Cat, Dog, Car).
 - **Location** → where the object is in the image (bounding box coordinates).
- Common use cases:
 - Autonomous driving (detecting pedestrians, cars, traffic signs).
 - Retail (detecting products on shelves).
 - Wildlife monitoring (detecting animals in camera trap images).
- Purpose: to train models that can recognize and locate multiple objects in complex scenes.

2. Task Setup

- **Define Object Categories:**
 - Example schema: Cat, Dog, Horse, Car, Person.
- **Annotation Tool Configuration:**
 - Import images into CVAT or another annotation platform.
 - Select **bounding box mode**.
- **Guidelines Document:**
 - Define how bounding boxes should be drawn:
 - Boxes must tightly fit the object (no excess background).
 - Boxes must cover the entire object (no missing parts).
 - Overlapping objects must each have their own box.
 - Clarify edge cases (e.g., partially visible objects, occluded objects).

3. Annotation Process

- **Step 1: Open the image** → identify all objects belonging to defined categories.
- **Step 2: Draw bounding boxes** → around each object.
- **Step 3: Assign correct label** → Cat, Dog, Car, etc.
- **Step 4: Double-check accuracy** → ensure boxes are proportional and consistent.

Examples:

- Image of a cat and a dog → two bounding boxes:
 - Box 1 → Cat
 - Box 2 → Dog
- Image of a street scene → multiple bounding boxes:
 - Car, Person, Traffic Sign.

4. Quality Assurance (QA) in Object Detection

- **Error Types:**
 - **Under-labeling** → missing objects or boxes too small.
 - **Over-labeling** → boxes including background or irrelevant objects.
 - **Mislabeling** → assigning wrong category (e.g., labeling a Dog as Cat).
 - **Inconsistent box size** → boxes too loose or too tight compared to guidelines.
- **Correction Steps:**
 - Review each image systematically.
 - Compare bounding boxes across similar objects.
 - Resize or redraw boxes to fit objects correctly.
 - Correct mislabels and ensure consistency.

5. Export and Validation

- **Export Format:** COCO JSON or Pascal VOC XML.
- **Validation Checks:**
 - Each bounding box must have coordinates [x, y, width, height].
 - Each box must be linked to the correct category.
 - No empty or invalid labels.

Example COCO JSON Export:

```
{  
  "images": [  
    {"id": 1, "file_name": "street_001.jpg", "width": 1920, "height": 1080}  
  ],  
  "annotations": [  

```

```
{ "id": 1, "image_id": 1, "category_id": 1, "bbox": [100, 200, 300, 400]},  
  { "id": 2, "image_id": 1, "category_id": 2, "bbox": [500, 600, 200, 300]}  
],  
"categories": [  
  { "id": 1, "name": "Car"},  
  { "id": 2, "name": "Person"}  
]  
}
```

Chapter: Image Segmentation (Detailed)

1. Definition and Purpose

- **Image Segmentation** assigns labels to pixels in an image, providing precise boundaries of objects.
- Unlike bounding boxes, segmentation captures the **exact shape** of objects.
- Segmentation can be divided into three main approaches:
 - **Semantic Segmentation** → classifies each pixel into a category, without distinguishing individual instances.
 - **Instance Segmentation** → classifies each pixel and separates different instances of the same category.
 - **Panoptic Segmentation** → combines semantic and instance segmentation into a unified representation.

2. Task Setup

- **Define Segmentation Schema:**
 - Example categories: Road, Car, Person, Tree, Building.
- **Annotation Tool Configuration:**
 - Import images into CVAT or similar tool.
 - Select polygon/mask mode depending on segmentation type.
- **Guidelines Document:**
 - Define rules for each segmentation type.
 - Clarify edge cases (e.g., occluded objects, overlapping categories).

3. Types of Segmentation

3.1 Semantic Segmentation

- **Definition:** Every pixel in the image is assigned a category label (e.g., Car, Road, Tree).
- **Limitation:** Does not distinguish between different instances of the same category.
- **Example:**
 - A street scene → all cars are labeled as “Car,” but individual cars are not separated.

3.2 Instance Segmentation

- **Definition:** Each pixel is assigned a category label **and** separated by instance.
- **Advantage:** Distinguishes between multiple objects of the same category.
- **Example:**
 - A street scene → Car #1 and Car #2 are labeled separately, even though both belong to the “Car” category.

3.3 Panoptic Segmentation

- **Definition:** A unified approach that combines semantic and instance segmentation.
- **Advantage:** Provides a complete pixel-level understanding of the scene.
- **Example:**
 - A street scene → Cars are labeled individually (instance), while Road and Sky are labeled as semantic categories.

4. Annotation Process

- **Step 1: Open the image** → identify all objects and regions.
- **Step 2: Draw polygons/masks** → carefully trace boundaries.
- **Step 3: Assign correct label** → according to segmentation type.
- **Step 4: Double-check accuracy** → ensure polygons are precise and consistent.

5. Quality Assurance (QA) in Segmentation

- **Error Types:**
 - **Under-segmentation** → missing parts of the object.
 - **Over-segmentation** → including background pixels.
 - **Mislabeling** → wrong category assignment.
 - **Inconsistent boundaries** → sloppy or jagged polygons.
- **Correction Steps:**
 - Review polygons systematically.
 - Compare boundaries across similar objects.
 - Redraw or refine polygons to fit objects correctly.
 - Correct mislabels and ensure consistency.

6. Export and Validation

- **Export Formats:**
 - COCO JSON (with segmentation field).
 - Mask images (binary or multi-class).
- **Validation Checks:**
 - Polygons must be represented as coordinate lists.
 - Masks must align with image dimensions.
 - Categories must match schema.

Example COCO JSON Export (Panoptic Segmentation):

```
{
  "images": [
    {"id": 1, "file_name": "street_001.jpg", "width": 1920, "height": 1080}
  ],
  "annotations": [
    {
      "id": 1,
      "image_id": 1,
      "category_id": 1,
      "segmentation": [[100,200,150,250,200,200]],
      "area": 2500,
      "iscrowd": 0
    }
  ],
  "categories": [
    {"id": 1, "name": "Car", "supercategory": "Vehicle"}
  ]
}
```


Chapter: Keypoint Annotation (Pose Estimation) (Detailed)

1. Definition and Purpose

- **Keypoint Annotation** is the process of marking specific points on an object (often a human or animal body) to represent its structure or pose.
- **Pose Estimation** uses these keypoints to understand body position, movement, or object geometry.
- Common use cases:
 - Human activity recognition (sports, fitness, gesture control).
 - Medical imaging (tracking joint positions).
 - Robotics (detecting hand or body pose for interaction).
 - Animation and AR/VR (motion capture).
- Purpose: to provide fine-grained information about object structure beyond bounding boxes or segmentation.

2. Task Setup

- **Define Keypoint Schema:**
 - Human pose → Head, Shoulders, Elbows, Wrists, Hips, Knees, Ankles.
 - Animal pose → Nose, Eyes, Tail, Paws.
 - Object geometry → Corners, Edges, Center points.
- **Annotation Tool Configuration:**
 - Import images into CVAT or similar tool.
 - Select **keypoint mode**.
- **Guidelines Document:**
 - Define exact placement rules (e.g., “Elbow keypoint must be placed at the joint, not on the arm”).
 - Clarify edge cases (e.g., occluded or partially visible joints).

3. Annotation Process

- **Step 1: Open the image** → identify the subject (human, animal, object).
- **Step 2: Place keypoints** → mark each defined joint or reference point.
- **Step 3: Assign correct label** → Head, Shoulder, Knee, etc.
- **Step 4: Connect keypoints (optional)** → draw skeleton lines to visualize pose.

- **Step 5: Double-check accuracy** → ensure keypoints are placed consistently across dataset.

Examples:

- Human pose image:
 - Keypoints → Head, Left Shoulder, Right Shoulder, Left Elbow, Right Elbow, Left Knee, Right Knee.
- Dog pose image:
 - Keypoints → Nose, Left Eye, Right Eye, Left Paw, Right Paw, Tail.

4. Quality Assurance (QA) in Keypoint Annotation

- **Error Types:**
 - **Missing keypoints** → forgetting to annotate a joint.
 - **Incorrect placement** → placing a keypoint on the wrong part of the body.
 - **Inconsistent labeling** → same joint labeled differently across images.
 - **Occlusion errors** → failing to mark partially visible joints.
- **Correction Steps:**
 - Review each image systematically.
 - Compare keypoint placement across similar poses.
 - Correct misplacements and ensure consistency.
 - Document edge cases for transparency.

5. Export and Validation

- **Export Format:** COCO JSON (with keypoints field).
- **Validation Checks:**
 - Each keypoint must have coordinates [x, y].
 - Visibility flags must be correct (0 = not labeled, 1 = labeled but not visible, 2 = labeled and visible).
 - Categories must match schema.

Example COCO JSON Export (Keypoints):

```
{
  "images": [
    {"id": 1, "file_name": "person_001.jpg", "width": 1920, "height": 1080}
```



```
],  
  "annotations": [  
    {  
      "id": 1,  
      "image_id": 1,  
      "category_id": 1,  
      "keypoints": [100,200,2, 150,250,2, 200,300,1],  
      "num_keypoints": 3  
    }  
  ],  
  "categories": [  
    {  
      "id": 1,  
      "name": "Person",  
      "keypoints": ["Head", "Left Shoulder", "Right Shoulder"],  
      "skeleton": [[1,2],[2,3]]  
    }  
  ]  
}
```

Chapter: Quality Assurance (QA) for Image Labeling (Detailed)

1. Definition and Purpose

- **Quality Assurance (QA)** in image labeling ensures that annotations are accurate, consistent, and aligned with project guidelines.
- Purpose:
 - Detect and correct errors before dataset export.
 - Guarantee reliability for training machine learning models.
 - Maintain consistency across annotators and tasks.

2. QA Workflow Overview

QA applies to all types of image labeling tasks:

- **Image Classification** → verifying correct category assignment.
- **Object Detection (Bounding Boxes)** → checking box placement and labels.
- **Segmentation (Polygons/Masks)** → validating pixel-level accuracy.
- **Keypoint Annotation (Pose Estimation)** → ensuring correct joint placement.

3. Common Error Types

- **Classification Errors:**
 - Misclassification (wrong category).
 - Inconsistent labeling across similar images.
- **Bounding Box Errors:**
 - Boxes too loose (include background).
 - Boxes too tight (cut off object parts).
 - Missing or duplicate boxes.
- **Segmentation Errors:**
 - Under-segmentation (missing pixels).
 - Over-segmentation (extra background).
 - Jagged or sloppy boundaries.
- **Keypoint Errors:**
 - Missing keypoints.
 - Incorrect placement (wrong joint).

- Inconsistent skeleton connections.

4. QA Process Steps

1. Guideline Review

- a. Ensure annotators follow project-specific rules.
- b. Check definitions of categories and edge cases.

2. Systematic Sampling

- a. Review a percentage of images (e.g., 10–20%) for errors.
- b. Use random and targeted sampling (complex images, occlusions).

3. Error Detection

- a. Compare annotations against guidelines.
- b. Identify mislabels, missing annotations, or inconsistent boundaries.

4. Correction

- a. Fix errors directly in the annotation tool.
- b. Document corrections for transparency.

5. Consistency Check

- a. Ensure similar objects/images are labeled the same way.
- b. Use inter-annotator agreement if multiple annotators are involved.

5. QA Metrics

- **Accuracy** → percentage of correctly labeled samples.
- **Consistency** → agreement across annotators.
- **Completeness** → all required objects/entities are annotated.
- **Precision of Boundaries** → segmentation polygons closely follow object edges.

6. Export and Validation

- **Export Formats:** COCO JSON, Pascal VOC XML, CSV.
- **Validation Checks:**
 - All images have labels.
 - Bounding boxes have valid coordinates.
 - Segmentation polygons are closed and valid.
 - Keypoints have correct coordinates and visibility flags.

Example QA Validation Report (Summary):

{

```
"total_images": 100,  
"reviewed_images": 20,  
"errors_found": {  
  "classification": 3,  
  "bounding_boxes": 2,  
  "segmentation": 4,  
  "keypoints": 1  
},  
"actions_taken": "Corrected mislabels, resized bounding boxes, refined segmentation  
polygons, adjusted keypoints."  
}
```

Chapter: Dataset Export and Structuring (COCO, Pascal VOC, etc.) (Detailed)

1. Definition and Purpose

- **Dataset Export and Structuring** is the process of saving annotated data into standardized formats used by machine learning frameworks.
- Purpose:
 - Ensure compatibility with training pipelines (PyTorch, TensorFlow, Detectron2, etc.).
 - Maintain consistency across projects.
 - Facilitate dataset sharing and validation.

2. Common Export Formats

2.1 COCO (*Common Objects in Context*)

- **Structure:** JSON file with sections:
 - `images` → metadata (id, file_name, width, height).
 - `annotations` → bounding boxes, segmentation, keypoints.
 - `categories` → list of classes.
- **Use Cases:** Object detection, segmentation, keypoint annotation.
- **Advantages:** Widely supported, flexible, handles multiple annotation types.

2.2 Pascal VOC

- **Structure:** XML file per image.
 - Contains image metadata and bounding box annotations.
- **Use Cases:** Object detection.
- **Advantages:** Simple, easy to parse, supported by classic models (e.g., Faster R-CNN).
- **Limitations:** Less flexible than COCO (no native support for keypoints or panoptic segmentation).

2.3 YOLO Format

- **Structure:** TXT file per image.

- Each line → `class_id x_center y_center width height` (normalized coordinates).
- **Use Cases:** Real-time object detection.
- **Advantages:** Lightweight, optimized for YOLO models.
- **Limitations:** Only bounding boxes, no segmentation or keypoints.

3. Export Workflow

1. **Complete Annotation** → ensure all images are labeled.
2. **Select Export Format** → COCO, Pascal VOC, YOLO depending on project.
3. **Export Dataset** → use annotation tool (CVAT, Labelbox, Roboflow).
4. **Validate Export** → check file structure, categories, and annotation consistency.
5. **Organize Dataset** → split into train/validation/test sets.

4. Validation Checks

- **File Integrity:** JSON/XML/TXT files must be valid and readable.
- **Category Consistency:** Labels must match schema.
- **Bounding Boxes:** Coordinates must be within image dimensions.
- **Segmentation Polygons:** Must be closed and valid.
- **Keypoints:** Must have correct coordinates and visibility flags.

5. Example Structures

COCO JSON Example:

```
{
  "images": [
    {"id": 1, "file_name": "dog_001.jpg", "width": 640, "height": 480}
  ],
  "annotations": [
    {"id": 1, "image_id": 1, "category_id": 1, "bbox": [100, 150, 200, 250], "iscrowd": 0}
  ],
  "categories": [
```

```
{ "id": 1, "name": "Dog" }
]
```

Pascal VOC XML Example:

```
<annotation>
  <folder>images</folder>
  <filename>dog_001.jpg</filename>
  <size>
    <width>640</width>
    <height>480</height>
  </size>
  <object>
    <name>Dog</name>
    <bndbox>
      <xmin>100</xmin>
      <ymin>150</ymin>
      <xmax>300</xmax>
      <ymax>400</ymax>
    </bndbox>
  </object>
</annotation>
```

YOLO TXT Example:

```
0 0.312 0.416 0.312 0.520
```

(Here 0 = class_id for Dog, followed by normalized coordinates.)

6. Dataset Structuring

- **Train/Validation/Test Split:**
 - Train → 70% of images.
 - Validation → 20% of images.
 - Test → 10% of images.
- **Folder Organization:**

dataset/

└─ train/

└─ val/

└─ test/

└─ annotations/

- **Documentation:**
 - Include README with dataset description, categories, and annotation guidelines.

Portfolio Highlights

Part I – Text Labeling

This portfolio demonstrates advanced skills in text annotation and quality assurance:

- **Text Classification** → Assigning categories such as sentiment (Positive/Negative/Neutral), spam detection, or customer support routing.
- **Named Entity Recognition (NER)** → Identifying and labeling entities (PERSON, LOCATION, ORGANIZATION, DATE, PRODUCT) with consistent schema.
- **Intent Classification** → Detecting the underlying purpose of queries (Travel Booking, Weather Query, Customer Support, Purchase).
- **Topic Labeling** → Categorizing texts into broader domains (Politics, Sports, Finance, Technology, Health).
- **Quality Assurance (QA)** → Reviewing annotations for accuracy, consistency, and completeness across all text tasks.
- **Export and Validation** → Delivering structured datasets in JSON/CSV formats, ensuring schema compliance and dataset integrity.

Part II – Image Labeling

This portfolio demonstrates advanced skills in image annotation and dataset structuring:

- **Image Classification** → Assigning categories to entire images (e.g., Cat, Dog, Horse, Product).
- **Object Detection (Bounding Boxes)** → Localizing and labeling multiple objects within an image.
- **Image Segmentation** → Precise pixel-level annotation using:
 - Polygons/Masks
 - Semantic Segmentation (class-level pixel labeling)
 - Instance Segmentation (separating individual objects of the same class)
 - Panoptic Segmentation (combining semantic + instance).
- **Keypoint Annotation (Pose Estimation)** → Marking joints or reference points for human/animal/object pose analysis.
- **Quality Assurance (QA)** → Detecting and correcting errors across classification, detection, segmentation, and keypoints.

- **Dataset Export and Structuring** → Delivering datasets in COCO JSON, Pascal VOC XML, and YOLO TXT formats, with proper train/validation/test splits and documentation.

Overall Competencies

- **Annotation Expertise** → Skilled in both text and image labeling tasks.
- **Quality Assurance** → Strong focus on accuracy, consistency, and dataset reliability.
- **Dataset Structuring** → Proficient in exporting and validating datasets across multiple formats.
- **Attention to Detail** → Ensuring edge cases are handled correctly and guidelines are strictly followed.
- **Versatility** → Capable of working across NLP and Computer Vision projects.